

Coding for Numbers

Diomidis Spinellis

Numerical Recipes in C++: The Art of Scientific Computing, 2nd edition by William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery, Cambridge University Press, 2002, ISBN 0-521-75033-4, 1,002 pp., US\$70.00.

Programs that perform sophisticated numerical calculations critically depend on the algorithms they use. Even if you obtain the correct requirements, come up with a brilliant architectural design, and write maintainable and efficient code, it's the underlying algorithm that will often make or break your application. The "Numerical Recipes" series of books distills in an accessible format the background theory, solid algorithms, and working programs for performing all but the most advanced numerical-analysis tasks needed for scientific computing and many engineering applications.

The book series has grown from the 1988 original Fortran-based book to include separate versions for Fortran 77, Fortran 90 (dealing with parallel computing), C, and C++. The second edition of *Numerical Recipes in C++: The Art of Scientific Computing* is more than 50 percent larger than the first and includes more than 300 algorithms.

Authoritative guidelines for complex calculations

The book's chapters detail linear algebraic equation solutions, interpolation and extrapolation, and functions' evaluation and minimization. They also cover Monte Carlo methods, root finding, Eigensystems, the Fourier transform and its applications, data modeling, boundary value problems, ordinary and partial differential equations, and integral equations. The book also contains other algorithms that scientists might find useful, such as those dealing with the statistical description of data, random

numbers, compression, coding, and arbitrary-precision arithmetic. The book's range of topics is extremely wide; hopefully, the next edition's minimization techniques discussion will include additional stochastic methods apart from simulated annealing, such as genetic algorithms and tabu search techniques.

Anyone performing calculations more complex than integer addition and subtraction is likely to benefit from this book's sometimes wry guidelines. If, for example, you think that $\sqrt{a^2 + b^2}$ is the way to find the Euclidean distance between two points, you'd better rush out for a copy of the book. The authors label this formula as "bad!" and explain that it will overflow if any of the numbers is as large as the square root of the largest representable number. Instead, they propose that you use

$$\begin{cases} |a|\sqrt{1 + \left(\frac{b}{a}\right)^2} & |a| \geq |b| \\ |b|\sqrt{1 + \left(\frac{a}{b}\right)^2} & |a| < |b| \end{cases}$$

In most cases, the authors don't formally examine the proposed algorithms' stability and accuracy properties—doing so would easily quadruple the book's size with minimal benefit to most readers. Instead, true to the book's title, the authors offer tried-and-tested recipes that will make your scientific programming a success.

Major improvements

The second edition finally does away with the

Fortran-derived arrays and matrices indexed from Element 1 that were infesting earlier editions and uses a number of C++ features to improve the algorithms' presentation and value. Most important, the authors use a family of templated objects to support a uniform implementation of variable-sized vectors and matrices. You can easily adapt the provided containers' minimal implementation into a wrapper for the more full-featured matrix library you might be using. In addition, the book's code isolates all program elements in appropriate C++ namespaces; efficiently passes arguments to functions using C++ references; uses the `const` keyword to identify function arguments that won't be modified; and declares small, heavily used functions as "inline."

Unfortunately, the reader expecting to find the C++ object-oriented and generic programming features applied on the algorithms themselves will be severely disappointed. The authors present all algorithms as global functions, still named with cryptic 1960s Fortran-era, six-character identifiers: `amotsa`, `bessk1`, `covsrt`, `erfcc`, and `frprmn`. They claim that this makes the functions easier to reuse in other applications, since the functions carry relatively little additional supporting baggage; the six-character identifiers maintain continuity and compatibility with the series' widely used earlier editions. However, I was left wondering how the power of full-fledged generic programming; the C++ Standard Template Library's containers, iterators, and algorithms; and the use of object-based information hiding, encapsulation, and inheritance could have been applied to the book's substantial and important body of algorithmic knowledge. Given that the book will be heavily used by scientists of all fields for years to come, its failure to use the abstractions and methods that we computing professionals consider important software building blocks is a problem that our profession should face, explain, and address.

Diomidis Spinellis is an associate professor in the Department of Management Science and Technology at the Athens University of Economics and Business. Contact him at dds@aub.gr.

ONLINE REVIEWS

- **"Uncovering Ignorance in Software Development"** by Paul Freedman
A review of *The Laws of Software Process: A New Model for the Production and Management of Software* by Philip G. Armour.
- **"A Definitive Source about UML"** by Wilson Pardi Jr.
A review of *UML Bible* by Tom Pender.

www.computer.org/software/bookshelf

A Book by Another Name

John R. Dance

Programming in the .NET Environment
by Damien Watkins, Mark Hammond,
and Brad Abrams, Addison-Wesley,
2002, ISBN 0-201-77018-0, 523 pp.,
US\$44.99.

A few years ago, I was working with an object framework whose base class provided a default identity-based `Hash()` and `Equals()`. I created a concrete subclass and wanted `Equals()` to compare the contents of my object, not just check for identity. I overrode `Equals()` and happily went on my way. Unfortunately, I had made a beginner's mistake. There's a strong relationship

between `Equals()` and `Hash()`. If two objects are "equal," the hash function must return the same value for both objects. If documentation is going to describe the `.NET System.Object` class, `Equals`, and `GetHashCode`, then it must necessarily describe this strong relationship. The `.NET Framework Software Development Kit` documentation passes this test. Unfortunately, *Programming in the .NET Environment* does not.

The book should actually be called *Common Language Runtime from the Perspective of a Compiler Writer* because that's what it covers best. Compilers and tools normally expose the CLR, so many people just think of it as their compiler runtime. This book skillfully highlights and demonstrates with concrete examples various pieces of the CLR. Key parts of the CLR are the type, metadata, and execution systems and how they interact across multiple languages. Each system gets an entire chapter in the book, which shows how both the CLR and base framework of `.NET` support building and deploying various types of applications. The book only broadly describes the framework class library, but, admittedly, a full description would take several volumes. About one-third of the book consists of appendices that describe, from the perspective of those that did the work, the porting of several language compilers to the `.NET` environment.

Although there's good material here, it can't overcome two major flaws—the unevenness of both the material and the editing. (These comments

The book should actually be called Common Language Runtime from the Perspective of a Compiler Writer because that's what it covers best.

don't pertain to the appendices, which were written by different authors about different languages.) The main chapters divide into thirds: the CLR proper, "programming in the large issues," and the framework. If I were a betting man, I'd say each author wrote one section. The framework section covers some of the same issues discussed in the type and metadata sections. The authors should have referred to the previous chapters rather than describe the issues again. Some of the work in the beginning of the book needed additional material, but perhaps it was dropped on the cutting-room floor. Often, introductory text describes the problems a construct is meant to solve, but then we dive down into the details so much that the solution is buried.

Careful editing should have resolved these and other issues. Other flaws include a paragraph that appears on a page owing to a cut-and-paste problem, references to source codes and directories we don't have (and have no easy way of locating without a URL), and definitions that are given once but never used again. Although the material is good, there are too many problems to ignore.

In retrospect, the text concentrates most on troublesome components of the .NET environment to a compiler porting effort. Perhaps a different title would have better reflected the book's content. Multiple examples and pages cover reflection and code generation through `System.Reflect.Emit` services. However, the section covering `System.Threading` is very brief. After giving a threading example, the text doesn't highlight the critical language constructs (C# lock statement) or classes (Mutex, Monitor) supporting synchronization. Instead, it says, "Be careful when dealing with multithreaded code, as each thread has access to the same state and the various threads' reads and writes to it can be interleaved." Unfortunately, I can only offer the same counsel about buying this book: "Be careful."

John R. Dance is a senior software engineer at PalmSource. Contact him at john.dance@palmsource.com.

Get Excited about Extreme Programming

David Arthur Eatough

A Practical Guide to Extreme Programming by David Astels, Granville Miller, and Miroslav Novak, Prentice Hall, 2002, ISBN 0-13-067482-6, 378 pp., US\$29.99.

When I first heard about Extreme Programming (XP) and unit testing, I was skeptical. At the time, I had more than enough code to write and debug. Taking time to write more code—even unit test code—just didn't make sense.

However, over the course of the last three years, I've learned how to unit-test. Today, I rely on my unit tests. They decrease the time from when I break something to when I know it's broken—reducing the time gap from days, weeks, or even months to mere minutes. Now, when code I modified breaks something, I immediately know that I need to change it. The obvious advantage is that the code is still fresh in my mind, so I can easily remember what I tried to do. Because of unit testing, I'm a more productive developer, and I produce higher-quality code.

But here's the drawback: How do you get to know and love unit testing without years of trial and error writing unit tests? Many books and papers are avail-

able about unit-testing theory. However, you'll find an invaluable example of a unit-tested application in *A Practical Guide to Extreme Programming*.

The book's most remarkable feature is its 122-page appendix, which explains the application and provides a complete source listing for it. The authors also explain how to download the program's source code, giving you a real-world example of a unit-tested application. You'll be able to roll up your sleeves, play with code, and see how unit testing can help you find problems that code changes introduce.

However, this book isn't just about unit testing. It also provides a great snapshot and insightful discussion about current XP principles and practices. The book discusses XP roles, including that of the architect. I agree with the authors that the project's architect should understand how to develop great software, but that rather than doing all the big upfront design per usual, he or she should look at the system as it evolves. The architect can then find the architecture's weaknesses and work with the team to improve the system.

The planning section contains a good summary of estimating and scheduling user stories. I particularly liked the concept of small development tasks, called "sand," which can be added when the development team finishes its stories early. This gives the team a quick win, and the customer also gets something of value. Too often, the developer-customer relationship is focused on what didn't get done. Many small wins throughout the development process can boost the morale of everyone involved.

Planning an XP project gets much harder with a large-team environment. It can be done, but it takes some real work. The authors talk openly about the challenges of scaling an XP project, including communication, coordination, and dealing with multiple sites. They recommend using several smaller teams, most of which will focus on one aspect of the project. However, some development teams will have to coordinate and collaborate with the customer on the smaller teams' work. Regardless of the ap-

Many books and papers are available about unit-testing theory. You'll find an invaluable example of a unit-tested application in *A Practical Guide to Extreme Programming*.

proach, be prepared to put more effort into organization and communication when using XP with a large team.

The authors discuss various options for gathering and capturing user requirements through user stories, use cases, and features. In every case, they noted: "If you can get the customer to ride alongside, both of you will enjoy the ride more." However, they could have provided more practical advice on what to do if your customer wants to use some of XP's features but still wants to give you requirements and come back later to see what has been accomplished, rather than riding along and providing feedback throughout the project's development.

Overall, I enjoyed *A Practical Guide to Extreme Programming*. It's true to its title, providing both a solid introduction and practical advice on using XP. Whether you're new to XP and want to understand its concepts or simply need some practical advice, this book will get you excited about XP.

David Arthur Eatough works for LANDesk Software. Contact him at deatough@acm.org.

IT Project Estimation: Recommended for Beginners

**Kevin C. Desouza
and Yukika Awazu**

IT Project Estimation: A Practical Guide to the Costing of Software by Paul Coombs, Cambridge University Press, 2003, ISBN 0-521-53285-X, 173 pp., US\$40.00.

IT Project Estimation discusses the art of creating, managing, and maintaining project estimates. Paul Coombs begins by explicating the nature of estimates, then discusses who should do the estimating, how they should do it, and how to review and continuously maintain estimates to gauge project performance. The book also covers important items such as analyzing project

risks. In the concluding chapters, Coombs provides an interesting case study that demonstrates the use of his guidelines. The book includes a supplemental CD containing various Microsoft Excel templates that you can use while conducting estimating exercises. Any project manager can use Coombs' guidelines regardless of industry, though they might be of greater value to IT projects.

The book covers concepts with an eloquent simplicity. The writing is easy to read and understand. We especially enjoyed Coombs's 12 "blindingly obvious rules of estimation"—every project manager should post these in his or her office and refer to them before beginning a project. Chapters are presented sequentially in the order of execution. The book also has a nice collection of references on specific estimation topics, such as Function Point Analysis. However, the book's key strength also contributes to its key weakness; it's a good read for a novice or aspiring project manager but offers little for a seasoned one.

Managing IT projects on time and within budget still remains an elusive goal. Various surveys have estimated that 30 to 65 percent of software development projects become runaway projects. Key reasons cited are inadequately specified objectives, bad planning and estimating, newness of the technology being implemented, inadequate project management methodol-

ogy, insufficient senior staff on the team, and poor performance by hardware or software suppliers.

Recently, we completed a study of why managers fail to detect signs of and pull the plug on out-of-control projects (Y. Awazu, K.C. Desouza, and J.R. Evaristo, "Stopping Runaway Information Technology Projects," *Business Horizons*, 2004). We found that IT projects are particularly difficult to manage and they have peculiarities that tend to blow project management problems out of proportion:

- They're riskier because the field is still immature. Managers must contend with a constant barrage of new products, tools, and technologies. Moreover, managing these tools and technologies isn't even close to being a science.
- IT projects face undue pressure in comparison to traditional projects, because many still view them as cost centers. They get bare-minimum budgets compared to other organizational endeavors, with little room for slack or redundancy. As a result, it doesn't take much for a project to start running away.
- IT projects aren't department- or group-specific; they typically affect multiple departments and involve multiple stakeholders. Sacrificing individual agendas to attain the global goal isn't an easy task.

All these problems have a central theme—poor estimation of efforts. Unless we have good estimates of the potential cost, time, and resource expenses, we won't be able to schedule, plan, manage, or track a project. We commend Coombs for contributing his thoughts on how we might move project estimation into the realm of science, and away from its current state of ad hoc art. ☞

Kevin C. Desouza is the director of the Institute for Engaged Business Research, a think tank of the Engaged Enterprise. Contact him at desouza@engagedenterprise.com.

Yukika Awazu is a research fellow at the Institute for Engaged Business Research. Contact her at awazu@engagedenterprise.com.

Unless we have good estimates of the potential cost, time, and resource expenses, we won't be able to schedule, plan, manage, or track a project.